

HUMAN AGENCY IN THE ERA OF AI-POWERED CONVERSATIONAL LARGE LANGUAGE MODELS: AN INTERPRETIVE SYNTHESIS OF DECISION-MAKING IN SOFTWARE ENGINEERING

Fasiha Ashraf^{*1}, Rashid Rasheed², Iqra Khalid³, Dr. Faiza Iftikhar⁴, Sakeena Parveen⁵

^{*1,2}Assistant Professor, Department of Computer Science, University of Management and Technology, Lahore, Pakistan

³Lecturer, Department of Computer Science, University of Management and Technology, Lahore, Pakistan

⁴Lecturer, Department of Electrical Engineering, Lahore College for Women University, Lahore, Pakistan

⁵MScS Scholar, Department of Computer Sciences, DHA SUFFA UNIVERSITY, Karachi, Pakistan

¹fasiha.ashraf@umt.edu.pk, ²rashid.rasheed@umt.edu.pk, ³iqra.khalid@umt.edu.pk,

⁴faiza.iftikhar@lcwu.edu.pk, ⁵sakeena.parveen1987@gmail.com

⁵<https://ORCID.org/0009-0001-4542-9914>

DOI: <https://doi.org/10.5281/zenodo.22294927>

Keywords

Artificial Intelligence; Human Agency; Large Language Models; Software Engineering; Human-AI Collaboration.

Article History

Received: 14 July 2026

Accepted: 18 August 2026

Published: 31 August 2026

Copyright @Author

Corresponding Author: *

Fasiha Ashraf

Abstract

The rapid integration of artificial intelligence (AI), generative AI, and conversational large language models (LLMs) has influenced software engineering by changing how developers generate, evaluate, and implement technical solutions. This study examined how these developments have shaped human agency in software engineering decision-making through a qualitative interpretive synthesis of existing secondary literature. Drawing on research concerning LLMs, AI coding assistants, human-AI collaboration, trust, automation bias, verification, expertise, and emerging agentic systems, the study synthesized evidence across heterogeneous scholarly sources. Four higher-order themes emerged: AI as an enabler of agency augmentation, negotiated agency through human-LLM interaction, agency constraints through trust and delegation, and the reconfiguration of expertise, responsibility, and control. The synthesis further identified three configurations of human agency: augmented agency, negotiated agency, and dependent agency. The findings suggested that AI could expand developers' capabilities while simultaneously creating risks of excessive reliance and reduced independent judgment. The study proposed an integrated framework emphasizing AI capability, interaction, trust, verification, expertise, task complexity, reliability, and organizational governance as key determinants shaping human agency.

1. Introduction

1.1 Background of Research

Artificial intelligence (AI) has increasingly transformed software engineering by shifting development activities from exclusively human execution toward human-AI collaboration. The evolution of transformer architecture established

the technical foundation for contemporary large language models (LLMs), enabling sophisticated natural language processing and generation (Vaswani et al., 2017). The subsequent emergence of foundation models expanded these capabilities across diverse knowledge-intensive tasks and professional domains (Bommasani et al., 2021).

Within software engineering, LLMs have increasingly been applied to requirements engineering, software design, coding, testing, debugging, maintenance, documentation, and project management (Hou et al., 2024). Alenezi and Akour (2025) further demonstrated the expanding role of AI-driven innovations in contemporary software engineering, while Banh et al. (2025) highlighted how generative AI was transforming software development practices through increasingly interactive and conversational forms of assistance.

AI coding assistants subsequently created new forms of interaction between developers and intelligent systems. Rather than functioning as simple automation, these systems enabled developers to generate, evaluate, modify, accept, or reject AI-produced solutions (Barke et al., 2023). Research on AI-assisted programming further showed that developers experienced both opportunities and challenges related to understanding, evaluating, and integrating AI-generated code into their workflows (Vaithilingam et al., 2022). Human-centered perspectives on AI-assisted programming emphasized that effective use of these systems requires continued developer judgment, oversight, and responsibility rather than passive acceptance of AI recommendations (Sarkar et al., 2022). More recent developments suggested a movement from AI assistants toward AI teammates and increasingly autonomous systems (Li et al., 2025; Roychoudhury et al., 2025).

These developments raised an important question concerning the position of human decision-makers. When AI generated suggestions, developers retained relatively clear decision authority; however, when AI systems generated substantial solutions or performed increasingly complex development activities, decision authority and responsibility became more distributed. The issue consequently extended beyond technological capability toward the human agency, autonomy, judgment, and control exercised during AI-mediated software engineering.

1.2 Problem Statement

Human agency provided an important theoretical perspective for understanding this transformation. Bandura (2001) conceptualized human agency through intentionality, forethought, self-reactiveness, and self-reflectiveness. In AI-assisted software engineering, these dimensions concerned whether developers continued to formulate intentions, evaluate alternatives, exercise professional judgment, and retain meaningful control over consequential decisions.

Existing research has generated substantial knowledge concerning AI capabilities, productivity, trust, security, developer interaction, and workflow transformation. For example, studies have examined how users calibrate trust, evaluate AI recommendations, and maintain human judgment in AI-assisted decision-making contexts (Liao et al., 2023), while research on AI-assisted development has highlighted security concerns and the continued importance of human oversight (Klemmer et al., 2024). However, this literature remained fragmented regarding the broader question of how AI participation collectively transformed human agency in software engineering decision-making.

Theoretical perspectives on hybrid problem-solving suggested that humans and AI could jointly contribute to organizational decision processes (Raisch & Fomina, 2025), while an assemblage perspective further indicated that agency could emerge through relationships between human and technological actors (Gehman et al., 2025). Melegati et al. (2024) similarly emphasized the human implications of intelligent tools within software engineering. Nevertheless, insufficient synthesis had connected these perspectives with evidence concerning trust, verification, expertise, delegation, responsibility, and emerging AI teammates. The present study therefore addressed this gap through a qualitative interpretive synthesis of existing secondary literature.

1.3 Research Questions

The study addressed the following research questions:

RQ1: How had AI-powered conversational LLMs transformed decision-making in software engineering?

RQ2: How had interaction with LLMs influenced human agency among software engineers?

RQ3: What roles had trust, verification, expertise, and AI reliability played in shaping human-AI decision-making?

RQ4: How had AI assistants and emerging AI teammates affected human control, responsibility, and decision authority?

RQ5: What configurations of human agency had emerged from AI-mediated software engineering practices?

1.4 Research Objectives

The study pursued five corresponding objectives:

RO1: To synthesize existing literature concerning the transformation of software engineering decision-making through conversational LLMs and generative AI.

RO2: To examine how interactions between software engineers and LLM-based systems had influenced human agency, autonomy, and decision-making.

RO3: To identify the roles of trust, verification, developer expertise, and perceived AI reliability in shaping human-AI decision-making.

RO4: To examine how AI assistants, AI teammates, and emerging agentic systems had redistributed decision authority, control, and responsibility within software engineering.

RO5: To develop an integrated conceptual understanding of the configurations of human agency emerging from AI-mediated software engineering.

1.5 Significance of the Study

The study was theoretically significant because it connected human agency theory with emerging research on generative AI, LLMs, and human-AI collaboration in software engineering. Rather than treating AI merely as a technological tool, it examined AI as an increasingly influential participant in decision-making processes. The study further distinguished between augmented, negotiated, and dependent agency, providing a

framework for understanding how human control could be strengthened, shared, or constrained through AI interaction.

Practically, the study provided implications for developers, organizations, AI designers, software engineering educators, and policymakers. Developers could benefit from greater emphasis on critical evaluation and verification, while organizations could use the findings to develop appropriate AI governance and accountability mechanisms. AI designers could incorporate features supporting transparency and human oversight, and educators could prepare future developers for AI-mediated development environments.

Methodologically, the study was significant because it was secondary, synthesizing existing scholarly literature rather than collecting primary data. Its scope remained focused on human agency and decision-making in AI-assisted software engineering, thereby providing an integrated understanding of an increasingly important but fragmented research area.

2. Literature Review and Theoretical Foundations

2.1 Evolution of LLMs and Generative AI in Software Engineering

The evolution of large language models (LLMs) represented a major technological shift in software engineering, moving AI from narrowly programmed automation toward systems capable of generating, interpreting, and transforming human language and code. The transformer architecture introduced by Vaswani et al. (2017) provided the fundamental architectural basis for contemporary LLMs by enabling more effective processing of contextual relationships within sequences. The subsequent development of foundation models expanded this capability beyond individual tasks, allowing large pretrained models to be adapted to multiple downstream applications (Bommasani et al., 2021). This evolution was particularly consequential for software engineering because programming languages share several structural and linguistic characteristics with natural language, making code

generation and code-related reasoning important applications of LLM technology.

The integration of LLMs into software engineering subsequently expanded beyond isolated code-generation experiments. Hou et al. (2024) identified applications across requirements engineering, software design, implementation, testing, debugging, maintenance, and project management, demonstrating that LLMs had become relevant across substantial portions of the software development lifecycle. Alenezi and Akour (2025) similarly described AI-driven innovation as increasingly embedded within contemporary software engineering practices. Thus, the role of AI shifted from performing narrowly defined computational functions toward supporting knowledge-intensive activities traditionally requiring substantial human expertise.

This progression subsequently produced conversational LLM interfaces through which developers could interact with AI using natural-

language prompts. Instead of learning a specialized command structure, developers could describe requirements, ask questions, request alternative solutions, or ask the system to explain generated code. Such conversational interaction altered the nature of human-AI engagement because developers increasingly became participants in iterative dialogues with AI systems. More recent developments have extended this trajectory toward AI teammates and autonomous coding agents. Li et al. (2025) described the emergence of AI teammates within what they characterized as a new stage of software engineering, while Bhati (2026) highlighted the increasing role of agentic AI throughout the software development lifecycle. Consequently, the evolution from conventional automation to agentic AI could also be interpreted as an evolution in the distribution of decision authority between humans and technological systems.

Table 1. Evolution of AI-Assisted Software Engineering

Stage	Technology	Dominant Human Role	Decision Character	Agency Implication
Traditional automation	Rule-based systems	Operator	Human-led	High human control
ML-assisted software engineering	Predictive models	Evaluator	Human-machine interaction	Augmented agency (Kellogg et al., 2020; Parent-Rochelleau & Parker, 2022)
LLM assistants	Conversational and code-generating LLMs	Prompting and verification	Interactive	Negotiated agency (Vaswani et al., 2017; Bommasani et al., 2021; Barke et al., 2023; Hou et al., 2024)
AI copilots	Integrated coding assistants	Reviewer and orchestrator	Collaborative	Reconfigured agency (Vaithilingam et al., 2022; Barke et al., 2023)
AI teammates	Agentic AI systems	Supervisor	Delegated	Potential dependent agency (Li et al., 2025; Roychoudhury et al., 2025; Bhati, 2026; Sergeyuk et al., 2026)

Note. Synthesized from the reviewed literature to show the progressive reconfiguration of human agency.

The trajectory presented in Table 1 suggested that technological advancement was accompanied by a gradual movement from direct human execution toward increasingly distributed decision-making. This transformation provided an important foundation for examining human agency in AI-supported software engineering.

2.2 AI-Assisted Software Development and Developer Workflows

The growing adoption of generative AI has transformed the activities through which software engineers conceptualize, produce, evaluate, and maintain software. AI coding assistants can generate code snippets, explain programming concepts, identify potential errors, suggest improvements, and support developers in navigating unfamiliar codebases. Barke et al. (2023) found that programmers interacted with code-generating models through different patterns of exploration and verification rather than simply accepting generated code. This demonstrated that AI-assisted programming involved an ongoing process of evaluating and negotiating machine-generated suggestions.

Research on GitHub Copilot further indicated that generative AI could improve developer productivity under particular conditions (Peng et al., 2023). However, productivity gains did not eliminate the need for human judgment. Sarkar et al. (2022) emphasized that AI-assisted programming requires developers to actively evaluate, interpret, and control AI-generated suggestions, illustrating that human judgment remains essential in determining whether AI outputs are appropriate for specific development tasks. Similarly, Zhang et al. (2024) demonstrated the expanding application of LLMs to automated program repair, indicating that AI was increasingly becoming involved in activities requiring analysis and modification of existing software.

The transformation also extended beyond code production. Wang et al. (2023) examined developers' trust in generative AI-assisted programming tools and highlighted the importance of validation, expectation management, and evaluation when interacting with AI-generated suggestions. Hou et al. (2024) further demonstrated that large language models are reshaping software engineering practices by introducing new forms of AI-assisted development while creating challenges related to human-AI interaction, evaluation, and responsible use. These findings suggest that AI should not be

conceptualized merely as another programming tool. Instead, it is increasingly becoming embedded within the cognitive and organizational processes through which developers make decisions.

This distinction was important for the present study. If AI merely accelerated a predetermined human decision, its effect on agency might remain limited. However, when AI generated alternatives, recommended solutions, explained trade-offs, or autonomously modified software, it potentially influenced the decision itself. Human agency therefore became dependent not only on whether developers could override AI recommendations but also on whether they continued to critically evaluate and understand the basis of those recommendations.

2.3 Human Agency and Human-AI Collaboration

Human agency provided the principal theoretical foundation for interpreting these changes. Bandura (2001) conceptualized agency as involving intentionality, forethought, self-reactiveness, and self-reflectiveness. Individuals were therefore viewed as capable of intentionally influencing their environments rather than simply responding to external forces. In software engineering, this perspective implied that developers' agency involved the ability to establish goals, select among alternatives, evaluate consequences, and exercise control over development decisions.

The increasing use of AI complicated this traditional human-centered understanding. AI systems could generate alternatives and contribute to problem-solving, meaning that decisions could emerge through interactions between human and technological actors. Raisch and Fomina (2025) conceptualized this relationship as hybrid problem-solving, emphasizing the complementary participation of human and artificial intelligence. Gehman et al. (2025) further advanced an assemblage perspective on hybrid agency, suggesting that agency could be understood through relationships among multiple actors

rather than being located exclusively within an individual.

This perspective was particularly relevant to software engineering because developers and AI systems frequently engaged in iterative cycles. A developer formulated a prompt, the LLM generated a response, the developer evaluated it, modified the prompt, requested alternatives, and potentially incorporated selected output into the software. Melegati et al. (2024) emphasized the importance of considering these human aspects when evaluating intelligent tools in software engineering. Agency could therefore become distributed or negotiated rather than completely transferred from humans to machines.

However, collaboration did not necessarily imply equal agency. Developers might retain formal authority while increasingly depending on AI recommendations. The distinction between formal control and substantive control was consequently important. A developer could theoretically reject an AI-generated solution but may become less capable of doing so effectively if AI systems increasingly shaped the available alternatives, assumptions, and technical decisions.

2.4 Trust, Automation Bias and Verification

Trust represented another central mechanism influencing human-AI decision-making. Developers needed to determine when AI-generated outputs were sufficiently reliable to accept and when additional verification was necessary. Mahmud et al. (2022) demonstrated that algorithmic decision-making could produce both algorithm aversion and algorithm appreciation, indicating that individuals' willingness to rely on algorithms depended on contextual and psychological factors. Jussupow et al. (2024) similarly emphasized that algorithm aversion and appreciation should not be understood as fixed attitudes but as dynamic responses to decision contexts.

Automation bias provided an important counterpoint to excessive trust. Lyell and Coiera (2017) showed that individuals could over-rely on automated recommendations, particularly when verification was difficult or cognitively demanding.

Although their systematic review focused on healthcare, the underlying mechanism was relevant to software engineering because developers may similarly accept plausible AI-generated outputs without independently validating every recommendation. This concern became particularly important where generated code appeared syntactically correct but contained security, logic, or architectural problems.

Recent research further connected these issues directly to AI-assisted software engineering. Liao et al. (2023) examined how users develop and calibrate trust in AI-assisted decision-making systems, highlighting the importance of critical evaluation and appropriate reliance when interacting with AI recommendations. Romeo and Conti (2026) likewise highlighted automation bias as a continuing issue in human-AI collaboration and emphasized the importance of explainability and appropriate oversight.

Verification therefore represented a critical mechanism through which human agency could be preserved. The developer who critically evaluated AI outputs retained a stronger role in decision-making than one who accepted recommendations with minimal scrutiny. Consequently, trust and verification could operate as competing forces: appropriate trust could facilitate productive collaboration, whereas excessive trust could increase dependence and weaken independent judgment.

2.5 Expertise, Control, Security and Emerging AI Teammates

AI adoption also transformed the relationship between expertise and control. Traditionally, software engineering expertise was associated with the ability to formulate solutions, write and evaluate code, diagnose problems, and make architectural decisions. AI-assisted development did not necessarily eliminate these capabilities but potentially changed how they were exercised. Developers increasingly needed to formulate effective prompts, evaluate generated outputs, identify hallucinations, verify security, and understand when AI-generated solutions were inappropriate.

The issue of control was also situated within broader organizational structures. Kellogg et al. (2020) demonstrated that algorithms could become a contested mechanism of organizational control, while Parent-Rochelleau and Parker (2022) showed how algorithms could influence the design and organization of work. These perspectives suggested that AI adoption could alter not only individual developer behavior but also organizational expectations concerning productivity, monitoring, task allocation, and decision authority.

Security represented another important dimension of human responsibility. Perry et al. (2023) demonstrated that AI coding assistants could potentially encourage the production of insecure code, reinforcing the importance of developer verification. Klemmer et al. (2024) similarly identified security practices and concerns surrounding AI assistants in software development. These findings suggested that AI-generated outputs could not simply be treated as authoritative technical solutions.

At the same time, AI systems were moving toward increasingly autonomous forms of participation. Roychoudhury et al. (2025) discussed agentic AI software engineers and the importance of trust when programming activities become increasingly autonomous. Hou et al. (2024) highlighted that generative AI in software development provides significant capability-enhancing opportunities while also introducing challenges related to human-AI interaction, evaluation, and responsible use. Sergeyuk et al. (2026) examined human-AI experiences within integrated development environments, while Treude and Storey (2025) further argued that generative AI represented a potential paradigm shift for empirical software engineering.

The movement from copilot to AI teammate therefore represented more than an increase in technological sophistication. It potentially involved a shift from AI providing suggestions toward AI participating in task planning, execution, evaluation, and coordination. As this transition continued, the developer's role could increasingly change from producer to reviewer,

orchestrator, or supervisor. Such developments raised fundamental questions regarding whether human agency would be augmented, redistributed, or progressively weakened.

2.6 Synthesis of the Literature and Research Gap

The reviewed literature collectively demonstrated that AI-assisted software engineering has evolved from rule-based automation toward increasingly interactive, generative, collaborative, and potentially autonomous systems. The technological literature established the expanding capabilities of LLMs across software engineering activities (Hou et al., 2024; Alenezi & Akour, 2025), while empirical studies highlighted changes in developer interaction, trust calibration, workflow adaptation, and experiences with AI-assisted programming (Barke et al., 2023; Vaithilingam et al., 2022; Wang et al., 2023). Research on human-AI collaboration further indicated that software development increasingly involves hybrid forms of problem-solving (Raisch & Fomina, 2025; Gehman et al., 2025).

However, the literature remains fragmented across technological, behavioral, organizational, and human factors perspectives. Studies of trust and automation bias have highlighted risks associated with excessive reliance on AI systems (Mahmud et al., 2022; Jussupow et al., 2024; Romeo & Conti, 2026), while security research has emphasized the continuing importance of verification and human responsibility (Perry et al., 2023; Klemmer et al., 2024). Research on AI teammates and agentic systems suggests that these considerations will become increasingly important as AI systems acquire greater autonomy (Roychoudhury et al., 2025).

The principal gap therefore concerns the absence of an integrated synthesis explaining how these developments collectively influence human agency in software engineering decision-making. Existing studies frequently examine individual dimensions such as productivity, trust, security, workflow, or AI capability. What remains insufficiently synthesized is how these dimensions interact to shape whether developers retain, share, or increasingly delegate decision authority.

The present interpretive synthesis addresses this gap by bringing together technological, behavioral, cognitive, and organizational evidence surrounding AI-assisted software engineering. In doing so, it examines human agency not as a fixed attribute but as a dynamic configuration shaped by AI capability, human-AI interaction, trust, verification, expertise, task demands, and organizational control. This synthesis provides the foundation for identifying different agency configurations and constructing the integrated human agency framework. Beyond trust and verification, cognitive engagement represents the mental effort through which developers interpret, challenge, and contextualize AI-generated outputs.

3. Methodology

3.1 Research Approach and Interpretive Synthesis Design

The study adopted a qualitative interpretive synthesis approach to examine how existing research had conceptualized human agency and decision-making in AI-assisted software engineering. The choice of a secondary research design was consistent with the purpose of the study, which was not to measure a new population or collect primary responses but to integrate findings from existing scholarly literature and develop a higher-order interpretation of fragmented evidence. An interpretive synthesis approach was particularly appropriate because the literature on generative AI in software engineering included empirical studies, qualitative investigations, systematic reviews, conceptual contributions, and emerging research on AI assistants and autonomous agents.

The study was informed by the meta-ethnographic principles of Noblit and Hare (1988), particularly the processes of identifying concepts across studies, comparing relationships among findings, and developing interpretations that extended beyond individual studies. Rather than simply aggregating findings, the interpretive synthesis sought to identify recurring conceptual relationships concerning human agency, AI reliance, decision-making, trust, verification, expertise, control, and responsibility. This

approach was important because the same phenomenon could have been described differently across software engineering, information systems, human-computer interaction, organizational studies, and AI research.

The reporting of the review process was guided by the PRISMA 2020 principles proposed by Page et al. (2021). PRISMA was used to structure the identification, screening, eligibility assessment, and final inclusion of literature, thereby improving transparency and methodological rigor in reporting the review process. However, the study remained fundamentally an interpretive synthesis rather than a statistical meta-analysis because the included literature differed considerably in research designs, contexts, theoretical perspectives, constructs, and outcome measures.

3.2 Literature Search Strategy

A structured literature search strategy was developed to identify scholarly studies examining the relationship between artificial intelligence, conversational large language models (LLMs), software engineering practices, and human decision-making. The search process was designed to capture multidisciplinary perspectives from computer science, software engineering, information systems, human-computer interaction, and organizational research. The literature search was conducted across five major academic databases: Scopus, Web of Science, IEEE Xplore, ACM Digital Library, and ScienceDirect. These databases were selected because they provide comprehensive coverage of peer-reviewed research in artificial intelligence, software engineering, computing, and human-technology interaction.

The search included publications from January 2017 to June 2026. The starting year was selected because the introduction of transformer architecture by Vaswani et al. (2017) represented a foundational advancement that enabled the development of modern large language models. The literature search was conducted in June 2026. The search strategy was organized around four

conceptual domains: artificial intelligence technologies, software engineering context, human decision-making and agency, and human-AI interaction mechanisms. The artificial intelligence domain included terms such as “large language model*,” “LLM*,” “generative AI,” “GenAI,” “conversational AI,” “AI assistant*,” “AI copilot*,” “AI teammate*,” and “agentic AI.” The software engineering domain included terms such as “software engineering,” “software development,” “programmer*,” and “developer*.” The human decision-making and agency domain incorporated terms including “human agency,” “autonomy,” “control,” and “decision-making,” whereas the human-AI interaction domain included concepts related to “trust,” “automation bias,” “verification,” “expertise,” “responsibility,” “delegation,” and “human-AI collaboration.”

Boolean operators were applied to combine these conceptual domains and identify relevant literature. The primary search string was: (*"large language model"* OR *"generative AI"* OR *"conversational AI"* OR *"AI assistant"* OR *"AI copilot"* OR *"agentic AI"*) AND (*"software engineering"* OR *"software development"* OR *programmer** OR *developer**) AND (*"human agency"* OR *autonomy* OR *control* OR *"decision-making"* OR *trust* OR *verification* OR *responsibility*). Additional searches were conducted using combinations of specific terms related to AI coding assistants, AI teammates, developer trust, human oversight, and autonomous AI-supported software engineering to ensure comprehensive identification of emerging literature relevant to the research objectives.

Because generative AI and agentic software engineering represent rapidly evolving research areas, recent conference papers and high-quality preprints were also considered where they provided substantive theoretical or empirical contributions.

3.3 Screening and Selection Procedure

The literature screening process followed the principles of PRISMA 2020 (Page et al., 2021) to ensure transparency and methodological rigor. The records retrieved from the selected databases were exported and screened through a sequential

process involving identification, duplicate removal, title and abstract screening, full-text assessment, and final inclusion.

A total of 70 records were initially identified through database searches. After removing one duplicate record, 69 studies remained for title and abstract screening. During this stage, studies were evaluated according to their relevance to artificial intelligence, conversational large language models, software engineering practices, human-AI interaction, and decision-making processes. Based on title and abstract screening, 29 studies were excluded because they did not directly address the research objectives or lacked sufficient relevance to human agency in AI-assisted software engineering. The remaining 40 articles were subjected to full-text assessment using predefined inclusion and exclusion criteria. Studies were included when they provided substantive contributions to understanding AI-assisted software engineering decision-making, human-AI collaboration, trust, verification, expertise, autonomy, responsibility, delegation, or human agency. Following full-text evaluation, 20 studies were excluded due to insufficient conceptual relevance, limited connection with software engineering decision processes, or inadequate contribution to the research questions. One additional study was subsequently removed because it substantially overlapped with another included publication and did not provide distinct analytical contribution.

Consequently, 19 studies were included in the final interpretive synthesis. The selection process emphasized conceptual relevance and theoretical contribution rather than statistical comparability, as the purpose of the study was conceptual integration and theory development rather than quantitative meta-analysis. The included studies represented diverse methodological approaches, including empirical investigations, conceptual studies, research syntheses, and systematic reviews. The screening and synthesis process involved iterative evaluation, comparison, and thematic interpretation of the selected studies. The authors continuously refined thematic categories by comparing findings across studies, identifying recurring concepts, and developing higher-order

constructs related to human agency transformation in AI-assisted software engineering. Any differences in interpretation were resolved through discussion and refinement of the coding structure.

3.4 Eligibility and Study Selection

Studies were selected using predetermined inclusion and exclusion criteria. The purpose was to ensure that the final evidence base remained directly connected to the research questions rather than simply reflecting general literature on artificial intelligence.

Inclusion criteria

Studies were included when they:

1. Examined AI, generative AI, LLMs, AI assistants, coding assistants, AI copilots, AI teammates, or agentic AI.
2. Addressed software engineering, software development, programming, or closely related development activities.
3. Examined at least one human-related dimension, including decision-making, agency, autonomy, trust, verification, expertise, control, responsibility, collaboration, or reliance.
4. Provided empirical, conceptual, theoretical, systematic review, or other substantial scholarly evidence.
5. Were published in English.
6. Were sufficiently accessible to allow interpretation of their principal arguments, methodology, and findings.
7. Contributed directly or conceptually to at least one research question.

Exclusion criteria

Studies were excluded when they:

1. Focused exclusively on AI technical architecture without relevance to human or software engineering issues.
2. Examined AI applications unrelated to software development.
3. Discussed education, healthcare, social media, or other application domains

without transferable relevance to software engineering decision-making.

4. Were opinion pieces, commercial promotional material, news articles, blogs, or non-scholarly webpages.
5. Duplicated evidence already represented by another publication.
6. Did not provide sufficient information to establish relevance to the research questions.

The screening process proceeded through identification, duplicate removal, title and abstract screening, full-text assessment, and final inclusion. The final number of records retained at each stage is reported in the PRISMA-based selection process.

3.5 Data Extraction and Quality Assessment

The final dataset consisted of 19 studies retained after screening, eligibility assessment, and removal of overlapping evidence. Each selected study was examined for its research context, AI application, principal human-AI issue, major insights, and contribution to understanding human agency. The extraction process allowed findings from technically oriented studies to be compared with findings from behavioral, organizational, and human-computer interaction research.

Given the methodological heterogeneity of the included literature, a standardized scoring approach was not applied. Instead, studies were critically appraised according to methodological relevance, conceptual clarity, and evidentiary contribution. Empirical studies were assessed according to the clarity of research design, sample or data source, analytical approach, and connection between evidence and conclusions. Conceptual studies were evaluated according to theoretical coherence, conceptual contribution, and relevance to the research questions. Research syntheses and systematic reviews were considered in terms of transparency, scope, synthesis approach, and relevance to the objectives of the present study.

Table 2

Characteristics and Analytical Contribution of Included Studies

Study	Context	AI Application	Human-AI Issue	Main Insight	Contribution to Human Agency
Barke et al. (2023)	Programmers using generative coding systems	Code-generating LLMs	Interaction, prompting, evaluation	Developers adopted different interaction strategies to explore, guide, and evaluate AI-generated code	Supports negotiated agency through continuous human-AI interaction and evaluation
Vaithilingam et al. (2022)	Programmers using AI code generation tools	AI-assisted programming systems	Usability, trust, limitations, developer experience	Developers valued AI assistance but identified challenges related to understanding, verification, and control of generated outputs	Supports augmented agency while emphasizing continued human oversight
Sarkar et al. (2022)	Human-centered AI-assisted programming environments	AI programming assistants	Collaboration, responsibility, human control	AI-assisted programming requires continued human involvement in reasoning, evaluation, and decision-making	Supports agency preservation through human-centered AI collaboration
Zhang et al. (2024)	Automated software repair research	LLM-based program repair	Automated code modification and decision support	LLMs expanded AI participation in software engineering activities while requiring human evaluation and supervision	Indicates changing boundaries between human and AI technical agency
Wang et al. (2023)	Software developers using AI code generation tools	AI-powered code generation systems	Trust calibration, validation, evaluation	Developers develop trust through experience, expectation management, and validation of AI-generated suggestions	Demonstrates negotiated agency through calibrated trust and responsible reliance
Melegati et al. (2024)	Software engineering professionals	Intelligent software tools	Human aspects of software engineering	Intelligent tools introduce new implications for professional practices, responsibilities, and developer roles	Provides conceptual foundation for understanding agency transformation

Study	Context	AI Application	Human-AI Issue	Main Insight	Contribution to Human Agency
Klemmer et al. (2024)	Software developers using AI development assistants	AI development assistants	Security, trust, verification	Developers highlighted security concerns and the importance of verifying AI-generated outputs	Reinforces human responsibility and oversight
Liao et al. (2023)	Users interacting with AI decision-support systems	Human-AI decision systems	Trust calibration, reliance, questioning AI outputs	Appropriate trust requires critical evaluation of AI recommendations rather than unconditional acceptance	Supports distinction between dependent and preserved agency
Hou et al. (2024)	Software engineering research and practice	Large language models for software engineering	AI capability, workflow transformation, human-AI challenges	LLMs represent a broad transformation of software engineering activities while introducing new human-centered challenges	Supports understanding of changing developer roles and AI-mediated agency
Banh et al. (2025)	Software engineering research	Generative AI	Collaboration and professional transformation	Generative AI contributes to changes in software engineering practices and workflows	Supports broader conceptualization of agency transformation
Treude & Gerosa (2025)	Software engineering professionals and AI-supported workflows	Generative AI systems	Human-AI interaction, collaboration patterns, developer roles	Human-AI collaboration requires reconsideration of developer practices, responsibilities, and interaction patterns	Supports negotiated and reconfigured agency
Raisch & Fomina (2025)	Organizational human-AI systems	Human-AI collaboration systems	Hybrid problem-solving	Human and AI capabilities can be combined to support new forms of problem-solving	Provides theoretical basis for hybrid agency
Gehman et al. (2025)	Human-AI collaboration research	AI systems	Distributed and relational agency	Agency may emerge through relationships between humans and intelligent systems rather than residing exclusively in humans or machines	Supports distributed agency perspective

Study	Context	AI Application	Human-AI Issue	Main Insight	Contribution to Human Agency
Roychoudhury et al. (2025)	Software engineering with autonomous AI agents	Agentic AI software engineers	Trust, autonomy, delegation	Increasing AI autonomy requires reconsideration of responsibility, trust, and human control	Supports delegated agency interpretation
Pearce et al. (2023)	Developers using AI-generated code systems	GitHub Copilot	Security, verification, responsibility	AI-generated code requires careful human review because automated suggestions may introduce security risks	Supports agency preservation through verification and human responsibility
Sergeyuk et al. (2026)	Integrated development environments	Human-AI IDE interaction	Experience, collaboration, interaction	AI increasingly becomes embedded within developer workflows and everyday technical activities	Supports reconfiguration of developer agency
Li et al. (2025)	Software engineering teams	AI teammates	Autonomous coding and collaboration	AI teammates represent an emerging stage of human-AI software development	Supports delegated and collaborative agency
Bhati (2026)	Software development lifecycle	Agentic AI	Autonomy and workflow	Agentic AI introduces new forms of AI participation across software development activities	Supports analysis of potential agency dependence
Treude & Storey (2025)	Empirical software engineering research	Generative AI	Research practices and professional roles	Generative AI represents an emerging shift in software engineering practices	Supports broader transformation of software engineering agency

Note. The table presents the principal characteristics and analytical contribution of the studies included in the final interpretive synthesis.

3.6 Coding and Interpretive Synthesis Procedure

The synthesis followed an iterative interpretive process involving three analytical levels. At the first-order level, concepts and findings were extracted using the terminology of the original studies. Examples included *AI assistance*, *developer verification*, *trust*, *productivity*, *automation bias*,

delegation, *security concerns*, and *human oversight*. At this stage, the extracted concepts represented the language and observations reported within the individual studies.

At the second-order level, conceptually similar findings were compared, grouped, and interpreted into broader thematic categories. Productivity

improvements, cognitive support, exploration, and expanded problem-solving capabilities were synthesized into the theme of AI assistance, representing the mechanism through which AI could support developers' capabilities. Similarly, prompting, refinement, evaluation, modification, and iterative interaction were grouped under human-AI interaction, reflecting the processes through which developers and AI systems jointly shaped software development decisions. Trust, delegation, automation bias, and reduced verification were synthesized into AI dependence, capturing conditions under which excessive reliance on AI could constrain independent judgment. Finally, verification, testing, and oversight were grouped under human control, representing mechanisms through which developers could preserve decision authority and accountability.

At the third-order level, relationships among the second-order themes were interpreted to generate higher-order constructs representing different configurations of human agency. The analytical pathway was therefore structured as:

First-order concepts → Second-order themes → Higher-order constructs → Integrated human-agency framework

The resulting synthesis identified four higher-order constructs: augmented agency, emerging from AI assistance and expanded cognitive capabilities; negotiated agency, emerging from continuous human-AI interaction and iterative decision processes; dependent agency, associated with excessive reliance, delegation, and reduced critical evaluation; and agency preservation, supported through verification, testing, and sustained human control. This process ensured that the final findings represented an interpretive synthesis across studies rather than a descriptive summary of individual publications.

3.7 Trustworthiness and Research Rigor

Several procedures were adopted to strengthen the credibility and consistency of the synthesis. First, the search and selection criteria were established

before final synthesis to reduce arbitrary inclusion. Second, a standardized extraction framework was applied across the literature. Third, findings were compared across different research designs and disciplinary perspectives rather than relying on a single type of evidence.

Interpretive consistency was further supported through constant comparison. Similar findings were compared across studies, while contradictory findings were retained and examined rather than automatically excluded. This was particularly important for issues such as trust and productivity because AI could simultaneously support performance improvements while creating risks of over-reliance.

This distinction enhanced interpretive validity by ensuring that conceptual extensions remained grounded in evidence relevant to AI-mediated software engineering. It also helped prevent inappropriate generalization of findings while allowing broader theoretical perspectives on agency, algorithms, and human-AI collaboration to inform the synthesis.

3.8 Ethical Considerations and Reporting

Because the study relied exclusively on previously published and publicly accessible scholarly literature, no human participants were recruited and no personally identifiable primary data were collected. Consequently, participant consent and institutional research approval were not required for this study. Nevertheless, ethical standards concerning accurate representation of previous scholarship, appropriate attribution, and avoidance of selective reporting were maintained throughout the synthesis.

The review process was reported in accordance with the principles of PRISMA 2020 (Page et al., 2021). The PRISMA framework was particularly useful for documenting how literature moved from initial identification through screening and eligibility assessment to final inclusion. The final selection numbers were derived from the completed search strategy, duplicate removal, screening, and eligibility assessment procedures.

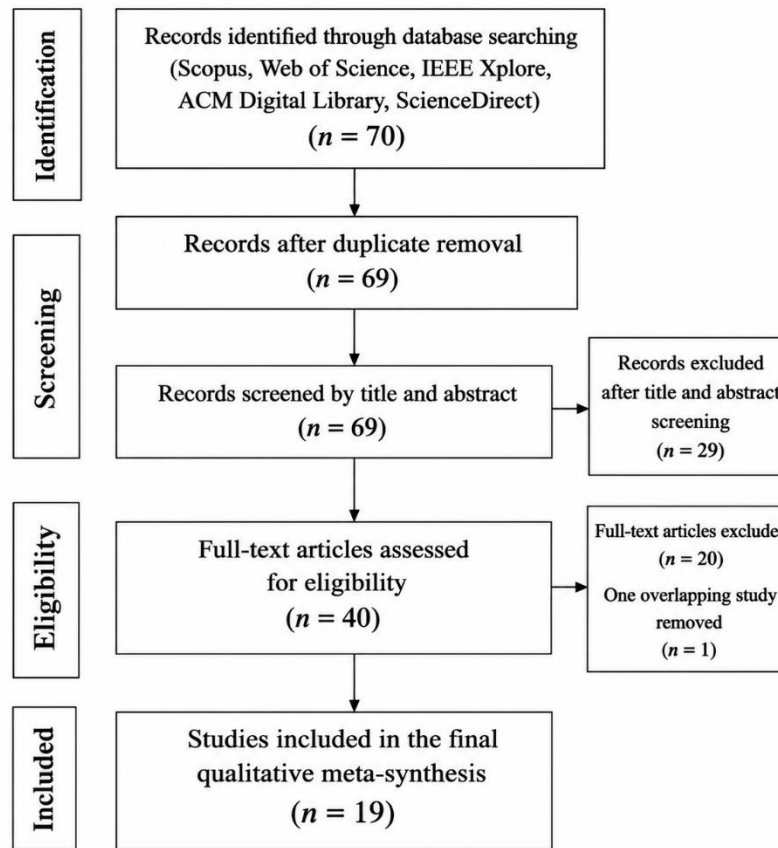


Figure 1. PRISMA-Based Literature Selection Process

The methodology consequently established a transparent and rigorous foundation for synthesizing secondary evidence on human agency in AI-powered conversational software engineering. By integrating systematic literature identification, screening, eligibility assessment, structured data extraction, and interpretive synthesis, the study moved beyond merely cataloguing AI applications and sought to explain the evolving nature of human decision authority, autonomy, responsibility, and control within AI-mediated software development environments.

4. Interpretive Synthesis Findings

The interpretive synthesis identified a complex evolution in the relationship between software engineers and AI-powered conversational systems. Across the reviewed literature, AI was not represented as producing a uniform effect on

human decision-making. Rather, its influence varied according to how developers interacted with AI, how much they trusted its outputs, the complexity of the task, the extent of verification, and the degree to which AI systems were permitted to undertake development activities autonomously. Four higher-order themes emerged from the synthesis: AI as an enabler of agency augmentation; negotiated agency through human-LLM interaction; agency constraints through trust, automation, and delegation; and reconfiguration of expertise, responsibility, and control. Integrating these themes resulted in three broader configurations of human agency: augmented agency, negotiated agency, and dependent agency.

4.1 AI as an Enabler of Agency Augmentation

The first major theme indicated that AI could support human agency by expanding developers'

cognitive and productive capabilities. Rather than replacing developers, AI-assisted tools frequently enabled them to explore more alternatives, complete routine activities more efficiently, and devote greater attention to complex aspects of software development. Barke et al. (2023) showed that programmers used code-generating models as interactive resources for exploring possible solutions, suggesting that AI could broaden the range of alternatives available to developers. Similarly, evidence from GitHub Copilot indicated that AI coding assistants provided cognitive support and influenced developer workflows, although effective use required interpretation and evaluation of AI-generated outputs (Vaithilingam et al., 2022).

The agency implications of this finding were important. Traditional software development required developers to independently formulate, search for, and implement solutions. AI-assisted development introduced new possibilities by reducing some of the cognitive and temporal costs associated with generating alternatives. A developer could ask an LLM to produce multiple implementation strategies, explain unfamiliar concepts, construct preliminary code, or identify possible solutions to a programming problem. These capabilities could increase the developer's capacity to act rather than necessarily diminishing it.

Banh et al. (2025) similarly identified generative AI as an important development in software engineering, with implications extending beyond isolated programming tasks. AI could influence broader workflows and facilitate collaboration between developers and intelligent systems. Alenezi and Akour (2025) likewise demonstrated the expanding role of AI-driven innovations across software engineering practices. Taken together, these findings suggested that AI had the potential to function as a capability multiplier.

However, agency augmentation did not mean that human agency automatically increased whenever AI was adopted. The benefits depended on the developer's ability to formulate appropriate requests, evaluate alternatives, and determine how AI-generated outputs should be used. AI therefore

appeared to expand the *potential* for agency while leaving the actual exercise of agency dependent on human judgment.

The synthesis consequently characterized this configuration as augmented agency, where AI expanded human capability without necessarily displacing human decision authority. Productivity gains, cognitive assistance, exploratory capacity, and access to alternative solutions represented the principal mechanisms through which this form of agency emerged.

4.2 Negotiated Agency Through Human-LLM Interaction

The second theme demonstrated that AI-assisted software engineering involved an ongoing process of negotiation between human intentions and machine-generated outputs. Developers did not simply issue commands and receive final solutions. Instead, they frequently prompted AI systems, examined outputs, identified shortcomings, reformulated requests, requested alternatives, and modified generated material before incorporating it into their work.

Barke et al. (2023) provided particularly important evidence for this pattern by demonstrating different ways programmers interacted with code-generating models. The interaction could involve exploratory prompting, iterative refinement, or evaluation of generated suggestions. Garousi and Jafarov (2025) similarly highlighted the importance of developer engagement with AI as part of a broader decision process rather than viewing AI adoption as simple automation.

This finding was consistent with the hybrid problem-solving perspective of Raisch and Fomina (2025), according to which human and artificial intelligence could jointly contribute to organizational problem-solving. In such situations, the resulting decision could not be attributed exclusively to either actor. The human established objectives and constraints, while AI generated alternatives or recommendations; the human subsequently evaluated those alternatives and potentially requested further AI input.

Wang et al. (2023) further highlighted that generative AI-assisted programming requires

developers to develop appropriate trust, evaluate AI-generated suggestions, and manage expectations regarding AI-supported development. Melegati et al. (2024) similarly emphasized the human aspects of intelligent tools in software engineering. Together, these studies suggest that the most significant change does not necessarily occur when developers surrender control to AI, but when they engage in continuous interaction, evaluation, and collaboration with AI systems.

Negotiated agency therefore differed from simple augmentation. Under augmentation, AI primarily expanded what the developer could accomplish. Under negotiated agency, AI also shaped the interaction process through which problems were framed and alternatives were considered. The developer remained involved in decision-making but operated within an expanded decision environment partially shaped by AI.

This form of agency could consequently be understood as relational. The developer's decision emerged from an interaction involving human expertise, AI-generated alternatives, contextual requirements, and iterative evaluation. Human agency remained present, but its expression was increasingly mediated by technology.

The synthesis therefore identified prompting, evaluation, refinement, rejection, modification, and iterative interaction as central mechanisms of negotiated agency. The quality of this configuration depended substantially on the developer's willingness and ability to challenge AI outputs rather than treating the conversational system as an unquestionable authority.

4.3 Agency Constraints Through Trust, Automation and Delegation

The third theme revealed the potentially constraining side of AI integration. Although AI could augment human capability, excessive trust and insufficient verification could constrain independent judgment. Mahmud et al. (2022) demonstrated that individuals could exhibit either algorithm aversion or algorithm appreciation depending on contextual factors. These contrasting responses were important because

both extremes could affect human decision-making. Excessive rejection could prevent useful AI support, while excessive acceptance could encourage inappropriate reliance.

Jussupow et al. (2024) provided broader evidence that responses to algorithmic recommendations vary according to decision context, suggesting similar mechanisms may influence AI-assisted software engineering. In software engineering, this suggested that developers' willingness to accept AI-generated code could vary according to task familiarity, perceived complexity, previous experience with AI, and confidence in the system. The literature on automation bias provided an additional explanation for potential constraints on agency. Lyell and Coiera (2017) demonstrated that people could accept automated recommendations without sufficient verification, particularly when checking the recommendation was difficult or costly. This mechanism was highly relevant to LLM-assisted programming because generated code could appear plausible, coherent, and technically sophisticated even when containing logical, security, or contextual problems.

Perry et al. (2023) provided evidence of this risk by demonstrating that AI-assisted coding could be associated with insecure code generation. Klemmer et al. (2024) similarly identified security practices and concerns surrounding AI assistants in software development. These findings highlighted that human oversight remained essential even when AI outputs appeared technically convincing.

Romeo and Conti (2026) further emphasized automation bias within human-AI collaboration and the importance of explainability and appropriate oversight. Liao et al. (2023) highlighted the importance of trust calibration in AI-assisted decision-making, demonstrating that effective human-AI interaction requires users to critically evaluate AI recommendations rather than rely on them unconditionally. Collectively, these studies suggest that trust operates as a double-edged mechanism. Appropriate trust can facilitate productive human-AI collaboration,

whereas excessive reliance may reduce critical evaluation and weaken human oversight.

The interpretive synthesis therefore identified trust, delegation, automation bias, and inadequate verification as mechanisms through which human agency could become constrained. When developers increasingly delegated cognitive tasks to AI without maintaining sufficient understanding or oversight, decision authority could remain formally human while becoming substantively dependent on AI recommendations.

This led to the identification of dependent agency as the third configuration of human agency. Dependent agency did not necessarily mean complete replacement of the human decision-maker. Instead, it represented a condition in which human judgment increasingly relied on AI-generated alternatives, recommendations, or outputs, potentially reducing independent reasoning and critical evaluation.

4.4 Reconfiguration of Expertise, Responsibility and Control

The fourth theme concerned the broader evolution of expertise, responsibility, and control as AI systems became increasingly integrated into software engineering. AI did not simply introduce another tool into an unchanged professional environment. Its adoption potentially influenced what developers needed to know, how work was allocated, and how responsibility for decisions was distributed.

Kellogg et al. (2020) demonstrated that algorithms could become mechanisms through which work and organizational control were restructured. Parent-Rochelleau and Parker (2022) similarly highlighted how algorithms could influence job design. Although these studies were not limited to generative AI in software engineering, their insights were relevant for understanding how AI could shape developers' work beyond individual interactions with coding assistants.

Treude and Gerosa (2025) highlighted that generative AI could influence software engineering practices by reshaping developer roles, workflows, and the nature of human-AI collaboration in software development. Pearce et

al. (2025) further emphasized that AI-assisted programming requires careful verification of AI-generated outputs, as developers remain responsible for evaluating correctness, security, and suitability before adopting AI-generated code. Sergeyuk et al. (2026) extended this discussion to human-AI experiences within integrated development environments, indicating that AI was becoming increasingly embedded within the everyday environments in which developers made technical decisions.

The emergence of AI teammates represented an especially important development. Li et al. (2025) described the rise of AI teammates as part of an emerging direction in software engineering, while Roychoudhury et al. (2025) examined the implications of agentic AI software engineers and the importance of trust in increasingly autonomous programming systems. Bhati (2026) similarly examined agentic AI across the software development lifecycle.

These developments suggested a gradual movement from **AI as assistant** → **AI as copilot** → **AI as teammate** → **AI as agent**. At each stage, the human role could evolve. Developers who previously produced code directly could increasingly become reviewers, orchestrators, supervisors, or strategic decision-makers. Consequently, expertise itself could shift from knowing how to manually execute every task toward knowing how to direct, evaluate, verify, and govern AI-supported work.

Responsibility represented another unresolved issue. If an AI-generated solution contained a security vulnerability or architectural error, the developer and organization could not simply attribute responsibility to the AI. Human oversight remained necessary. Thus, increasing AI capability did not eliminate human responsibility; instead, it potentially created more complex forms of shared or mediated responsibility.

The synthesis consequently indicated that AI was reconfiguring rather than simply reducing human control. The central question became whether developers could maintain sufficient expertise and oversight as AI systems assumed increasingly complex responsibilities.

4.5 Cross-Theme Synthesis: Three Configurations of Human Agency

The four higher-order themes were integrated to identify three configurations through which human agency appeared to operate in AI-mediated software engineering. These configurations were

not mutually exclusive stages through which every developer necessarily progressed. Rather, they represented different conditions that could emerge depending on AI capability, task complexity, developer expertise, trust, verification, and organizational context.

Table 3. Interpretive Synthesis of Human Agency Configurations

Evidence-Level Concepts	Synthesized Theme	Agency Configuration	Main Implication
Assistance, productivity, cognitive support, exploration, alternative generation, problem-solving support, task acceleration	AI augmentation	Augmented agency	AI can expand developers' cognitive and productive capacity while human decision authority remains substantially intact
Prompting, questioning, contextualizing, refinement, evaluation, modification, rejection, iterative interaction	Human-AI negotiation	Negotiated agency	Human and AI contributions interactively shape alternatives and decisions, creating shared and relational decision-making
Trust, delegation, automation bias, over-reliance, insufficient verification, acceptance of plausible outputs	AI dependence	Dependent agency	Increasing reliance on AI recommendations may constrain independent judgment, critical evaluation, and meaningful human control
Verification, testing, expertise, critical evaluation, code review, security assessment, human oversight	Human oversight	Agency preservation	Active verification and human expertise support decision authority, accountability, and meaningful control over AI-supported outcomes

Note. The table synthesized first-order concepts into higher-order agency configurations identified across the reviewed literature.

The first configuration, augmented agency, occurred when AI supported developers' capabilities while human decision authority remained substantially intact. The second, negotiated agency, emerged when decisions resulted from iterative interaction between developers and AI systems. The third, dependent agency, emerged when developers increasingly relied upon AI-generated recommendations and delegated substantial cognitive responsibility to the system.

The fourth row in Table 3, agency preservation, represented an important cross-cutting mechanism rather than a separate configuration. Verification, testing, expertise, and critical evaluation functioned as safeguards capable of

preventing augmentation or negotiation from developing into excessive dependence.

These configurations suggested that human agency in AI-assisted software engineering existed on a dynamic continuum rather than as a binary human-versus-machine condition. The same developer could experience augmented agency during one task, negotiated agency during another, and dependent agency when working under time pressure or with unfamiliar technical domains. AI capability alone could not determine the level of human agency. Instead, agency emerged through the interaction between technological capability and human practices.

4.6 Summary of Interpretive Synthesis Findings

The interpretive synthesis demonstrated that AI-powered conversational systems have influenced software engineering decision-making in four interconnected ways. First, AI expanded the potential for human capability by supporting productivity, exploration, and access to potential solutions. Second, developers are increasingly engaged in negotiated interaction with LLMs through prompting, evaluation, refinement, and modification. Third, trust, automation bias, delegation, and inadequate verification created potential constraints on independent human judgment. Finally, the movement toward AI teammates and agentic systems was associated with evolving forms of expertise, responsibility, and control within software development.

The synthesis consequently established three principal configurations of human agency: augmented agency, in which AI supported human capability; negotiated agency, in which humans and AI jointly shaped decisions; and dependent agency, in which human judgment increasingly relied on AI recommendations. Verification, testing, expertise, and human oversight emerged as critical mechanisms for maintaining agency across these configurations.

These findings provided the empirical-conceptual foundation for the next stage of the study. Chapter 5 therefore builds on this synthesis to develop an integrated human agency framework explaining how AI capability, human-AI interaction, trust, expertise, verification, task characteristics, and organizational governance collectively shape whether software engineers experience augmented, negotiated, or dependent agency.

5. Integrated Framework and Theoretical Contributions

5.1 Development of the Integrated Human Agency Framework

The interpretive synthesis demonstrated that the relationship between software engineers and AI-

powered conversational systems could not be adequately explained through a simple replacement or augmentation perspective. The findings suggested that human agency in AI-assisted software engineering was shaped through a combination of technological capability, interaction patterns, trust, expertise, verification, delegation, and organizational conditions. The integrated framework was therefore developed to explain how these factors collectively shaped different configurations of human agency during AI-assisted software engineering.

The framework was derived directly from the four higher-order themes identified in Chapter 4. AI capability represented the technological starting point because increasingly capable LLMs expanded the range of software engineering activities that could be supported or delegated. Human-AI interaction shaped how developers engaged with these capabilities through prompting, evaluation, refinement, and collaboration. Trust, expertise, and verification represented critical human-level mechanisms influencing whether AI recommendations were critically assessed or readily accepted. These mechanisms subsequently influenced the degree of decision delegation, which contributed to the resulting configuration of human agency.

The framework conceptualized agency as dynamic and relational rather than fixed. Accordingly, developers could experience augmented agency when AI supported their capabilities while they retained substantial decision authority; negotiated agency when human and AI contributions jointly shaped decisions; and dependent agency when substantial reliance on AI recommendations could constrain independent judgment. The framework further recognized that task complexity, AI reliability, and organizational governance could influence movement between these configurations.

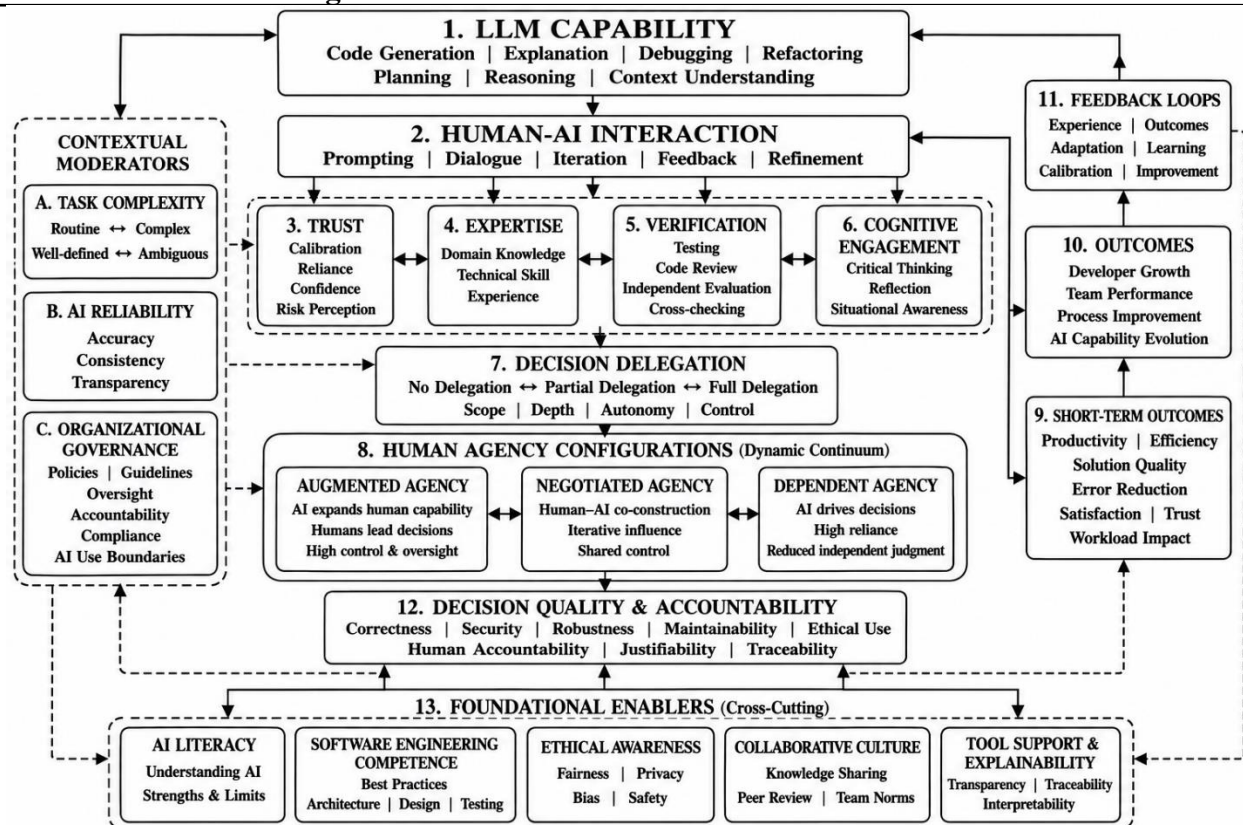


Figure 2. Integrated Human Agency Framework for AI-Assisted Software Engineering: Mechanisms, Configurations, Contextual Conditions, and Outcomes

5.2 Core Mechanisms of Agency Transformation

Eight mechanisms emerged as particularly important in explaining variations in human agency.

LLM capability represented the technological capacity of AI to generate code, explain solutions, identify errors, develop alternatives, and increasingly support sequences of software engineering tasks. Greater capability expanded the range of activities potentially available for AI support or delegation (Hou et al., 2024; Alenezi & Akour, 2025).

Human-AI interaction shaped how those capabilities were translated into practice. Iterative prompting, evaluation, refinement, and modification enabled developers to remain actively involved in decision-making (Barke et al., 2023; Wang et al., 2023).

Trust shaped how developers evaluated and relied on AI recommendations. Appropriate trust could

facilitate productive human-AI collaboration, whereas excessive reliance on AI recommendations could increase dependence and reduce critical evaluation (Mahmud et al., 2022; Liao et al., 2023).

Verification represented the principal safeguard against inappropriate AI reliance. Testing, code review, security assessment, and independent evaluation enabled developers to maintain meaningful oversight (Klemmer et al., 2024; Pearce et al., 2025).

Expertise influenced developers' capacity to understand, evaluate, and challenge AI-generated outputs. As AI supported more routine coding activities, expertise increasingly involved knowing how to evaluate and govern AI-supported work rather than simply producing code manually.

AI reliability affected the extent to which delegation could occur safely. Higher perceived reliability could encourage greater reliance, while

uncertainty or inconsistent outputs could encourage stronger human oversight.

Task complexity shaped the consequences of delegation. Routine and well-defined tasks could potentially be delegated more easily, whereas complex architectural, security-sensitive, or ambiguous decisions required stronger human involvement.

Organizational governance influenced how individual developers interacted with AI. Organizational policies concerning accountability, security, review procedures, permitted AI use, and human oversight could either preserve or constrain individual agency.

5.3 Three Agency Configurations

The framework conceptualized human agency as a dynamic continuum involving three principal configurations.

Augmented Agency

Augmented agency emerged when AI expanded human capability without substantially displacing human decision authority. Developers used AI to generate alternatives, accelerate routine activities, explore solutions, and overcome information constraints while maintaining responsibility for evaluating and implementing outputs. Evidence concerning productivity, exploration, and developer interaction supported this configuration (Barke et al., 2023; Vaithilingam et al., 2022).

Negotiated Agency

Negotiated agency emerged when software engineering decisions were produced through iterative interaction between humans and AI. Developers shaped AI outputs through prompts and contextual information, while AI contributed to the alternatives available to developers. The resulting decision was therefore jointly shaped rather than exclusively human or machine generated. This configuration reflected the hybrid problem-solving perspective proposed by Raisch and Fomina (2025).

Dependent Agency

Dependent agency emerged when developers increasingly delegated cognitive or technical decision-making to AI and relied heavily on AI-generated recommendations. Trust, time pressure, limited expertise, low verification, and high perceived AI reliability could increase this tendency. Automation bias represented an important mechanism through which dependence could become problematic (Lyell & Coiera, 2017; Romeo & Conti, 2026).

The three configurations were not necessarily permanent developmental stages. Rather, the same developer could move between them according to task characteristics, AI reliability, expertise, workload, and organizational expectations. Thus, the framework proposed dynamic movement rather than a deterministic progression from augmentation toward dependence.

5.4 Role of Human Oversight and Organizational Governance

The synthesis interpreted that existing literature suggests emerging configurations of human agency could not depend exclusively on individual developer competence. Even highly skilled developers operated within organizational environments that shaped the extent and manner of AI use. Organizational pressure for productivity, standardized AI workflows, automated deployment, or extensive delegation could influence developers' opportunities to independently verify AI-generated outputs.

Human oversight therefore needed to operate at multiple levels. At the individual level, developers required sufficient expertise to evaluate AI outputs. At the team level, peer review, testing, and security procedures could provide additional safeguards. At the organizational level, governance mechanisms could establish accountability, acceptable-use policies, documentation requirements, and escalation procedures.

This multi-level perspective was consistent with research demonstrating that algorithms could influence organizational control and work design (Kellogg et al., 2020; Parent-Rocheleau & Parker,

2022). As AI systems became more autonomous, governance became increasingly important for ensuring that delegation did not develop into uncontrolled dependence.

5.5 Theoretical Propositions

The integrated framework generated the following theoretical propositions:

Proposition 1: Greater LLM capability may increase opportunities for human agency augmentation in software engineering by expanding developers' cognitive and productive capacity.

Proposition 2: Greater human-AI interaction through prompting, evaluation, and refinement is expected to be associated with stronger forms of negotiated agency.

Proposition 3: Appropriate verification of AI-generated outputs may strengthen human control and reduce the likelihood of dependent agency.

Proposition 4: Excessive trust in AI-generated recommendations may increase the likelihood of decision delegation and dependent agency.

Proposition 5: Higher developer expertise may strengthen the capacity to critically evaluate AI outputs and preserve human agency.

Proposition 6: Greater task complexity may increase the need for human involvement and reduce the appropriateness of unrestricted decision delegation to AI.

Proposition 7: Higher perceived AI reliability may increase willingness to delegate software engineering decisions to AI, with this relationship moderated by verification and developer expertise.

Proposition 8: Strong organizational AI governance may strengthen human oversight and reduce the risk that AI-assisted decision-making develops into excessive dependence.

5.6 Theoretical Contributions

The study made four principal theoretical contributions. First, it extended human agency theory by conceptualizing agency in AI-assisted software engineering as dynamic and relational. Rather than treating agency as an exclusively individual human characteristic, the framework showed how agency could be augmented, negotiated, or constrained through interaction with AI.

Second, the study contributed to human-AI collaboration research by demonstrating that collaboration should not automatically be interpreted as beneficial augmentation. Human-AI collaboration could generate different agency configurations depending on trust, verification, expertise, and delegation. This extended hybrid agency perspectives advanced by Raisch and Fomina (2025) and Gehman et al. (2025).

Third, the study contributed to software engineering research by connecting technical developments in LLMs, coding assistants, and AI teammates with questions of decision authority, responsibility, and developer control. Existing research had extensively documented what AI could do; the framework additionally addressed what AI adoption could imply for the human actor responsible for software decisions.

Fourth, the study contributed to AI governance by positioning human oversight as an agency-preservation mechanism rather than merely a compliance requirement. Governance was conceptualized as a structural condition affecting whether developers could maintain meaningful control over AI-supported decisions.

5.7 Practical Implications

For software developers, the findings emphasized the importance of maintaining verification skills, questioning AI outputs, understanding limitations, and avoiding uncritical acceptance of generated solutions. AI literacy should therefore include evaluation and oversight rather than prompt-writing alone.

For organizations, AI adoption should be accompanied by clear governance structures specifying responsibility for AI-generated code, review requirements, security procedures, and appropriate levels of delegation. Productivity objectives should not eliminate opportunities for meaningful human verification.

For AI designers, systems should support rather than obscure human judgment. Explainability, uncertainty indicators, traceability, verification support, and mechanisms for human intervention could help developers maintain agency.

For software engineering educators, curricula should increasingly incorporate AI-assisted development alongside critical evaluation, code verification, security assessment, prompt formulation, AI literacy, and responsible decision-making.

For policymakers, the findings highlighted the need for governance approaches that preserve meaningful human accountability as software engineering becomes increasingly AI-mediated. Regulations and organizational standards should recognize that human oversight becomes particularly important when AI systems move from generating recommendations toward executing development activities autonomously.

Overall, the framework demonstrated that the central challenge was not determining whether AI should participate in software engineering, but determining how its participation could expand human capability without progressively displacing meaningful human judgment and accountability. The distinction between augmented, negotiated, and dependent agency provided a conceptual basis for addressing this challenge.

6. Discussion, Conclusion and Future Research

6.1 Discussion of the Interpretive Synthesis

The interpretive synthesis demonstrated that human agency in AI-assisted software engineering had not been eliminated by the emergence of conversational large language models; rather, it had been reconfigured through interaction with increasingly capable AI systems. Four interconnected findings explained this evolution. First, AI could augment human capability by

increasing productivity, exploration, and access to alternative solutions. Second, human-AI interaction increasingly involved negotiation through prompting, evaluation, refinement, and revision. Third, excessive trust, automation bias, and insufficient verification could constrain independent judgment. Fourth, the increasing movement toward AI teammates and agentic systems was associated with evolving forms of expertise, responsibility, and control.

The first finding was consistent with evidence that coding assistants could improve developer productivity and support exploration of programming alternatives (Barke et al., 2023; Vaithilingam et al., 2022). However, the synthesis extended this productivity perspective by demonstrating that increased productivity did not necessarily constitute a complete change in agency. Agency depended on whether developers remained capable of evaluating and controlling the resulting outputs. Thus, the contribution of AI was better understood as an expansion of potential human action rather than automatic replacement of human action.

The second finding supported research describing software development as an increasingly interactive human-AI process. Developers could prompt systems, examine generated solutions, reject inappropriate outputs, and request refinements (Barke et al., 2023; Wang et al., 2023). This finding was also compatible with hybrid problem-solving perspectives, which conceptualized human and artificial intelligence as potentially complementary contributors to decision processes (Raisch & Fomina, 2025). The present synthesis extended this argument by introducing negotiated agency as a specific configuration in which human decisions were shaped through repeated interaction, evaluation, and collaboration with AI systems.

The third finding highlighted an important tension. AI could simultaneously support and constrain agency. Trust could facilitate adoption and productive interaction, but excessive trust could encourage inappropriate delegation. Previous literature on algorithm aversion and appreciation had demonstrated that individuals

responded differently to algorithmic recommendations (Mahmud et al., 2022; Jussupow et al., 2024). In software engineering, this issue became particularly significant because AI-generated code could appear technically credible while containing security or functional problems (Pearce et al., 2025; Klemmer et al., 2024). Verification therefore emerged as a central mechanism separating productive AI reliance from problematic dependence.

Finally, the synthesis indicated that the transition from coding assistants toward AI teammates and agentic systems could contribute to broader changes in software engineering work. Research on AI teammates and agentic software engineers suggested that AI was increasingly moving beyond isolated code suggestions toward more autonomous participation in development activities (Li et al., 2025; Roychoudhury et al., 2025). Consequently, developers could increasingly function as reviewers, orchestrators, supervisors, and governance actors rather than solely as direct producers of code.

6.2 Theoretical Interpretation of Human Agency Evolution

The findings extended Bandura's (2001) social cognitive theory by applying an agentic perspective to a technological environment in which humans increasingly acted with and through intelligent computational systems. Bandura's conception of human agency emphasized intentionality, forethought, self-reactiveness, and self-reflectiveness. These dimensions remained relevant in AI-assisted software engineering, but their practical expression was evolving as developers could delegate portions of cognitive work to conversational AI.

The study therefore suggested that contemporary software engineering agency should not be conceptualized solely as an individual's independent capacity to act. Instead, it could be understood as the capacity to direct, evaluate, negotiate, and govern AI-mediated action. This interpretation complemented hybrid-agency perspectives emphasizing the combination of human and artificial capabilities (Raisch &

Fomina, 2025). Gehman et al. (2025) similarly emphasized that agency could emerge through relationships between humans and technological systems.

The distinction among augmented, negotiated, and dependent agency represented the principal theoretical extension of the study. Augmented agency preserved human decision authority while expanding capability. Negotiated agency recognized that AI increasingly participated in shaping the alternatives considered by human decision-makers. Dependent agency identified the condition in which human decision-making became substantially reliant upon AI recommendations or outputs.

This framework therefore moved beyond a binary distinction between human control and machine control. It proposed that agency could vary according to the degree of interaction, delegation, verification, expertise, and organizational governance. Such an interpretation was particularly important as AI systems became increasingly capable and autonomous.

6.3 Implications for AI-Assisted Software Engineering

The findings carried implications for several stakeholders. For developers, AI literacy needed to extend beyond prompt engineering toward critical evaluation, verification, security assessment, and understanding of model limitations. Developers who retained strong technical expertise could use AI as an augmentation mechanism while maintaining greater control over decisions.

For organizations, AI adoption required governance mechanisms that established responsibility for AI-generated outputs. Development organizations needed appropriate review procedures, security checks, documentation standards, and boundaries around autonomous AI activity. This was particularly important because algorithms could influence the organization and design of work itself (Kellogg et al., 2020; Parent-Rochelleau & Parker, 2022).

For AI designers, systems needed to support meaningful human oversight rather than encourage passive acceptance. Features supporting

uncertainty, communication, traceability, verification, testing, and human intervention could help preserve agency.

For software engineering education, curricula needed to prepare students for AI-mediated development environments. Future developers would require not only programming knowledge but also the ability to evaluate AI outputs, detect errors, assess security implications, and make informed decisions about delegation.

For AI governance, the findings suggested that human oversight should be treated as an active organizational capability rather than a symbolic requirement. As AI systems moved toward more autonomous forms of execution, accountability mechanisms needed to ensure that humans retained meaningful authority over consequential software decisions.

6.4 Answer to the Research Questions

RQ1: How had AI-powered conversational LLMs influenced decision-making in software engineering?

The synthesis indicated that LLMs had influenced decision-making by expanding access to coding alternatives, explanations, debugging assistance, and problem-solving support. Decision-making had increasingly become interactive and AI-mediated rather than exclusively human-driven (Hou et al., 2024; Banh et al., 2025).

RQ2: How had human agency been affected by AI-assisted software engineering?

Human agency had generally shifted from direct execution toward augmentation, negotiation, evaluation, and supervision. AI could strengthen agency when developers retained control, but agency could become constrained when decision authority was increasingly delegated to AI.

RQ3: What factors influenced whether AI strengthened or constrained human agency?

The principal factors were LLM capability, human-AI interaction, trust, verification, developer expertise, AI reliability, task complexity, and organizational governance. These factors

collectively shaped the extent to which developers remained actively engaged in decision-making.

RQ4: How did trust, verification, and delegation influence developer decision-making?

Trust facilitated AI use, but excessive trust could encourage over-reliance. Verification acted as an agency-preserving mechanism by allowing developers to identify inappropriate outputs. Greater delegation could increase efficiency in suitable circumstances but could weaken independent judgment when verification and expertise were insufficient (Lyell & Coiera, 2017; Klemmer et al., 2024).

RQ5: How did the transition toward AI teammates and agentic systems affect human responsibility and control?

The transition introduced greater complexity in human responsibility. Developers increasingly moved toward roles involving orchestration, supervision, evaluation, and governance. Consequently, human responsibility remained important even when AI assumed greater operational autonomy (Roychoudhury et al., 2025; Li et al., 2025).

6.5 Overall Contributions of the Study

The study made theoretical, practical, and methodological contributions. Theoretically, it extended human agency theory into AI-mediated software engineering by proposing three configurations of agency: augmented, negotiated, and dependent. It also connected individual agency with hybrid human-AI collaboration and organizational governance.

Practically, it provided a framework through which organizations could evaluate whether AI adoption was expanding developer capability or creating excessive dependence. The framework emphasized verification, expertise, and governance as mechanisms for preserving meaningful human control.

Methodologically, the study demonstrated the value of qualitative interpretive synthesis for integrating heterogeneous literature across software engineering, AI, human-computer

interaction, organizational research, and decision-making. Instead of treating these bodies of research independently, the synthesis connected them around the common issue of human agency.

6.6 Limitations

Several limitations required consideration. First, the study was entirely secondary, meaning that its conclusions depended on the quality, scope, and interpretation of existing research. No primary observations or developer interviews were conducted to independently validate the synthesized configurations.

Second, the literature was heterogeneous. Studies differed in methodological design, participant characteristics, AI systems, software engineering tasks, and theoretical perspectives. Although interpretive synthesis enabled meaningful conceptual integration, such heterogeneity limited direct comparison across individual findings.

Third, generative AI and agentic software engineering were developing rapidly. Findings that appeared current during the review could potentially become outdated as models, development environments, and organizational practices evolved. The inclusion of recent emerging research helped address this issue but could not eliminate it.

Finally, the interpretive nature of the synthesis meant that the resulting framework represented a theoretically grounded interpretation rather than a statistically tested causal model. The proposed relationships and agency configurations therefore require empirical validation.

6.7 Future Research and Conclusion

Future research should empirically examine the proposed framework across different software engineering environments. Quantitative studies could examine relationships among AI capability, trust, verification, expertise, delegation, and the three agency configurations. Longitudinal research would be particularly valuable because developers' reliance on AI could change as familiarity and system capability increased.

Future studies should also compare novice and expert developers to determine whether expertise

protects against excessive AI dependence. Experimental research could investigate how verification behavior changes under different levels of task complexity, time pressure, and perceived AI reliability. Research should additionally examine agentic AI systems capable of planning and executing multi-step development activities rather than merely generating code.

At the organizational level, future research should investigate how governance structures influence human agency and accountability. Comparative research across organizations could determine whether formal AI policies, review procedures, and security controls reduce the emergence of dependent agency.

In conclusion, this interpretive synthesis demonstrated that the central issue surrounding AI-powered conversational LLMs in software engineering was not whether humans or AI would control development. Instead, the more important issue was how human agency would evolve as increasingly capable AI systems became embedded within software engineering decisions. AI could augment human capability, reshape decision-making through negotiation, or create dependence when trust and delegation exceeded verification and expertise. The proposed framework therefore positioned augmented, negotiated, and dependent agency as three complementary configurations through which the future relationship between software engineers and AI could be understood. Preserving meaningful human agency would require maintaining a balance between AI capability and human judgment through expertise, verification, accountability, and effective organizational governance.

REFERENCES

- Adhami, N., & Moore, R. L. (2026). Intelligent tutoring systems in language learning: A systematic review of pedagogical design and learner impact. *TechTrends*, 70(1), 183–199.

- Agarwal, A., Sebastian, M. P., & Krishnan, S. (2026). Toward a better digital society: A meta-synthesis on generative AI's role in disinformation creation and mitigation. *Internet Research*, 36(3), 1093–1112.
- Alenezi, M., & Akour, M. (2025). AI-driven innovations in software engineering: A review of current practices and future directions. *Applied Sciences*, 15(3), 1344.
- Amangeldy, B., Tasmurzayev, N., Imankulov, T., Baigarayeva, Z., Izmailov, N., Riza, T., ... Zhumagulov, B. (2025). AI-powered building ecosystems: A narrative mapping review on the integration of digital twins and LLMs for proactive comfort, IEQ, and energy management. *Sensors*, 25(17), 5265.
- Babar, Z., Paul, R., Rahman, M. A., & Barua, T. (2025). A systematic review of human-AI collaboration in IT support services: Enhancing user experience and workflow automation. *Journal of Sustainable Development and Policy*, 1(1), 65–89.
- Bandura, A. (2001). Social cognitive theory: An agentic perspective. *Annual Review of Psychology*, 52, 1–26.
- Banh, L., Holldack, F., & Strobel, G. (2025). Copiloting the future: How generative AI transforms software engineering. *Information and Software Technology*, 183, 107751.
- Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 85–111.
- Bhati, H. (2026). Agentic AI in the software development lifecycle: Architecture, empirical evidence, and the reshaping of software engineering. *arXiv preprint arXiv:2604.26275*.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., ... Liang, P. (2021). *On the opportunities and risks of foundation models*. arXiv.
- Choudhuri, R., Trinkenreich, B., Pandita, R., Kalliamvakou, E., Steinmacher, I., Gerosa, M., Sanchez, C., & Sarma, A. (2025). What guides our choices? Modeling developers' trust and behavioral intentions towards GenAI. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering* (pp. 1691–1703). IEEE.
- Costa, A. P., Bryda, G., Christou, P. A., & Kasperuniene, J. (2025). AI as a co-researcher in the qualitative research workflow: Transforming human-AI collaboration. *International Journal of Qualitative Methods*, 24, 16094069251383739.
- Garousi, V., & Jafarov, Z. (2025). *How software engineers engage with AI: A pragmatic process model and decision framework grounded in industry observations*. arXiv.
- Gehman, J., Glaser, V. L., & Merritt, P. (2025). An assemblage perspective on hybrid agency: A commentary on Raisch and Fomina's "Combining human and artificial intelligence." *Academy of Management Review*.
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., ... & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 1-79.
- Jussupow, E., Benbasat, I., & Heinzl, A. (2024). An integrative perspective on algorithm aversion and appreciation in decision-making. *MIS Quarterly*, 48(4), 1575–1590.

- Kellogg, K. C., Valentine, M. A., & Christin, A. (2020). Algorithms at work: The new contested terrain of control. *Academy of Management Annals*, 14(1), 366–410.
- Klemmer, J. H., Horstmann, S. A., Patnaik, N., Ludden, C., Burton, C., Powers, C., Massacci, F., Rahman, A., Votipka, D., Lipford, H. R., Rashid, A., Naiakshina, A., & Fahl, S. (2024). Using AI assistants in software development: A qualitative study on security practices and concerns. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2726–2740). Association for Computing Machinery.
- Koç, H., & Uzun, A. M. (2025). Exploring studies on the use of artificial intelligence in education: A meta-synthesis study. *Erzincan Üniversitesi Eğitim Fakültesi Dergisi*, 27(3), 409–426.
- Li, H., Zhang, H., & Hassan, A. E. (2025). The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*.
- Liao, Q. V., Gruen, D., & Miller, S. (2023). Questioning the AI: A human-centered approach to understanding trust in AI-assisted decision making. *Proceedings of the 2023 ACM Conference on Human Factors in Computing Systems (CHI)*. Association for Computing Machinery.
- Lyell, D., & Coiera, E. (2017). Automation bias and verification complexity: A systematic review. *Journal of the American Medical Informatics Association*, 24(2), 423–431.
- Machado, C., & Davim, J. P. (Eds.). (2026). *Artificial intelligence technologies in management and engineering*. John Wiley & Sons.
- Mahmud, H., Islam, A. K. M. N., Ahmed, S. I., & Smolander, K. (2022). What influences algorithmic decision-making? A systematic literature review on algorithm aversion. *Technological Forecasting and Social Change*, 175, 121390.
- Melegati, J., Nascimento, N., Chanin, R., Sales, A., & Wiese, I. (2024). Exploring potential implications of intelligent tools for human aspects of software engineering. In *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)* (pp. 121–132). Association for Computing Machinery.
- Milakis, E. D., Argyrakou, C. C., Melidis, A., & Vrettaros, J. (2025). ChatGPT and AI chatbots in education: An umbrella review of systematic reviews, scoping reviews, and meta-analyses. *International Journal of Education and Information Technologies*, 19, 100–119.
- Nguyen-Trung, K., & Nguyen, N. L. (2026). Narrative-integrated thematic analysis (NITA): How can LLMs support theme generation without coding? *Qualitative Research in Psychology*, 1–37.
- Noblit, G. W., & Hare, R. D. (1988). *Meta-ethnography: Synthesizing qualitative studies*. SAGE Publications.
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., ... Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *Systematic Reviews*, 10(1), Article 89.
- Parent-Rochelleau, X., & Parker, S. K. (2022). Algorithms as work designers: How algorithmic management influences the design of jobs. *Human Resource Management Review*, 32(3), 100838.

- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2025). Asleep at the keyboard? assessing the security of github copilot's code contributions. *Communications of the ACM*, 68(2), 96-105.
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv*.
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2785-2799). Association for Computing Machinery.
- Raisch, S., & Fomina, K. (2025). Combining human and artificial intelligence: Hybrid problem-solving in organizations. *Academy of Management Review*, 50(2), 441-464.
- Romeo, G., & Conti, D. (2026). Exploring automation bias in human-AI collaboration: A review and implications for explainable AI. *AI & Society*, 41, 259-278.
- Roychoudhury, A., Pasareanu, C., Pradel, M., & Ray, B. (2025). *Agentic AI software engineers: Programming with trust*. *arXiv*.
- Sarkar, A., Gordon, A. D., Negreanu, C., Poelitz, C., & Zorn, B. (2022). The human in the loop: Exploring the role of human judgment in AI-assisted programming. *Proceedings of the 2022 ACM Conference on Human Factors in Computing Systems (CHI)*. Association for Computing Machinery.
- Sergeyuk, A., Zakharov, I., Koshchenko, E., & Izadi, M. (2026). Human-AI experience in integrated development environments: A systematic literature review. *Empirical Software Engineering*, 31(3), 55.
- Treude, C., & Gerosa, M. A. (2025, April). How developers interact with AI: A taxonomy of human-AI collaboration in software engineering. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)* (pp. 236-240). IEEE.
- Treude, C., & Storey, M. A. (2025). Generative AI and empirical software engineering: A paradigm shift. In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)* (pp. 233-239). IEEE.
- Ulfesnes, R., Moe, N. B., Stray, V., & Skarpen, M. (2024). Transforming software development with generative AI: Empirical insights on collaboration and workflow. In *Generative AI for effective software development* (pp. 219-234). Springer.
- Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022, April). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts* (pp. 1-7).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. *arXiv*.
- Wang, C., Chen, Y., Hu, Z., Li, Y., & Gu, X. (2025). The journey of challenges and victories: Exploring the transformation action framework in the GenAI era from multifaceted policies. *Educational Technology Research and Development*, 73(5), 2951-2993.
- Wang, J., & Li, Y. (2026). AI and the future of educational technology: A systematic review of pedagogical innovations and challenges. *Cogent Education*, 13(1), 2691562.

- Wang, R., Cheng, R., Ford, D., & Zimmermann, T. (2024, June). Investigating and designing for trust in ai-powered code generation tools. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency* (pp. 1475-1493).
- Yang, Y., & Traube, D. (2026). AI in mental health services. In *Artificial intelligence in social work* (pp. 171-191). Springer.
- Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Practices and challenges of using GitHub Copilot: An empirical study. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering* (Vol. 2023-July, pp. 124-129). Knowledge Systems Institute Graduate School.
- Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Yang, Y., & Chen, Z. (2024). A systematic literature review on large language models for automated program repair. arXiv.
- Zhang, W., Tangi Perrault, S., Yeo, S., Pei, J., De Liddo, A., Veri, F., ... Saggion, H. (2026). AI agents and the future of deliberation: Designing human-AI collaboration for democratic dialogue. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems* (pp. 1-6).